

IndexedEx / DTF — security re-audit

Date: 2026-09-17 **Commit audited:** `aca16198` (crane `d87c74b7`) **Chain:** Robinhood mainnet, chainId 4663 **Auditor:** will - zauth (Vector APEX v1.23.5)

Safety statement, up front

Everything below was executed on a **local fork**. Against the live chain I made **read-only `eth_calls` only** — no transaction was ever broadcast, and I never took custody of any at-risk funds. The open window described in §1 has been left exactly as found.

TL;DR

#	Finding	Severity	Deployed?
001-M	<code>_secureTokenTransfer</code> credits tokens never transferred	Critical	LIVE — open now
001-M2	Scope: 7 vaults carry it, not 1	—	LIVE
003	<code>takeShares</code> refunds to <code>msg.sender</code> , not the depositor	High	LIVE
008	Orbital hook pays out its entire face balance	High	not yet
009	<code>_burnSeShares</code> sweeps all resting shares to the caller	High	not yet
004B	Idle underlying swept to the caller	High	not yet
005	Root cause: no canonical unbooked-balance helper	High	—

Two findings from my earlier message **do not survive** and I am retracting them — see §6. I would rather correct myself than have you spend time on a non-issue.

One change fixes most of this. See §5.

1. APEX-2026-001-M — Critical, live right now

`UniswapV4StandardExchangeCommon.sol:1270` (and V3 `:855`, byte-identical):

```

uint256 R          = MultiAssetBasicVaultRepo._reserveOfToken(address(tokenIn)); // STORED
uint256 B0         = tokenIn.balanceOf(address(this));
uint256 deployed   = _deployedFaceOf(address(tokenIn)); // LIVE
uint256 faceBooked = R > deployed ? R - deployed : 0;
uint256 U          = B0 > faceBooked ? B0 - faceBooked : 0;
if (amountIn > U) revert ISecurePullErrors.TransferDeltaInsufficient(amountIn, U);
return amountIn; // credited WITHOUT any transfer

```

R is written at the last `_updateReserve`. `deployed` is recomputed every call from the current pool tick. Differencing a **live** quantity against a **stale** one is not a conservative floor — the error is unbounded in whichever direction the pool has moved.

When the pool moves such that `deployed > R`, the clamp yields `faceBooked == 0`, so:

```
U = B0 - 0 = the entire free balance
```

...credited to the caller with no transfer.

Measured on a fork of your mainnet, real deployed bytecode, block 64025200. An actor starting with zero tokens, calling only permissionless functions, ended holding **127.42 vault shares**. Broke in 1 call.

Live state at that block, vault `0x999DaE02D22E5FEe1c4508430D5196d31d631009`:

```

DTF  leg: B0 647,486.246   R 32,374,312.30   deployed 37,345,542.62
      faceBooked clamps to 0 -> U = B0 = 647,486.246 DTF      ** OPEN **

WETH leg: B0 17.3146      R 43.6682         deployed 22.3886
      faceBooked 21.2795 > B0 -> U = 0                                closed
      opens if deployed0 rises 3.9650 WETH (+17.7%)

```

The rebalance is not a fix

`rebalanceLiquidReserve()` re-pins **R** at a moment in time. The pool keeps moving afterwards and the gate reopens on its own. Two independent confirmations:

- Observed reopening within **~2 hours** of your 12:02 rebalance.
- Replaying the gate over real chain history (archival `eth_call`, 8 samples spanning 5,250 blocks): **open in 8/8**, every one the clamped case. **R** did not change once across the entire window while `deployed` changed every block.

That difference *is* the bug. Any mitigation that re-syncs **R** buys minutes to hours.

2. APEX-2026-001-M2 — it is not one vault

You mentioned this was one vault with the factory kept hidden. From the registry `0x09682b00...` : **49 vaults, 7 carry this code path, 2 currently open**. They span DTF, PONS and MARTIANS:

<code>0x999DaE02D22E5FEe1c4508430D5196d31d631009</code>	UniV4 Vault of (WETH / DTF)	OPEN
<code>0xb7D4Bb379D361AD442CddEE53eA71a33957826A7</code>	UniV4 Vault of (WETH / DTF)	
<code>0x3E33871A8740b294A48347BFFD8A5004E1578c0D</code>	UniV4 Vault of (WETH / PONS)	
<code>0x201d31a56c58a3489beC218c7A899d59A07A35f0</code>	UniV4 Vault of (WETH / PONS)	
<code>0xabC9C5B5c8118a15A440Ac59a1C74A718E65681C</code>	UniV4 Vault of (WETH / PONS)	
<code>0x17B5f1045Edf3b6215d0C38eB9FF50649d887D10</code>	UniV4 Vault of (USDG / MARTIANS)	
<code>0x64e0f5A5Cf0F579EFF0981A4fEDEA77964B74531</code>	UniV4 Vault of (ETH / PONS)	

Reproduce with `reserveOfToken(token) + deployedReserve() + balanceOf(vault)` per leg. Note the correct selector for `reserveOfToken(address)` is `0x46f910ac` — I initially used a wrong one and got a false "4 vaults open", so it is worth double-checking against a known-good `cast` value.

3. APEX-2026-003 — High, live

Custody vault `0x2E9C1F705aB967c5Af59249203d7495bDabb223b` ,
`RebasingAwareERC4626Common.takeShares` :

```
uint256 prepaid = ERC20Repo._balanceOf(address(this));
ERC20Repo._burn(address(this), amount);
uint256 refund = prepaid - amount;
ERC20Repo._transfer(address(this), msg.sender, refund); // msg.sender, not the depositor
```

The refund goes to whoever **calls**, not to whoever pretransferred. The two-step pretransfer flow (`transfer` then `exchange`) is therefore front-runnable: a victim moves shares onto the vault, an attacker calls first with a minimal burn and takes the remainder.

Fork PoC confirms. Note a 1-wei burn trips `ZeroOperationOutput` at the $\sim 1e10:1$ share ratio — the attacker needs the smallest burn yielding non-zero output ($\sim 1e11$); everything above it still comes back as "refund".

This vault correctly reverts `assetIn && pretransferred`, so it does **not** carry 001-M. Different primitive, same root cause.

4. Pre-deployment findings — worth fixing before these ship

APEX-2026-008 — orbital buffer hook, High

`UniswapV4StandardExchangeOrbitalBufferHookCommon.sol:1988` :

```

function _freeTokenBalance(address token) internal view returns (uint256 free) {
    uint256 bal = IERC20(token).balanceOf(address(this));
    if (_seOf(token) != address(0)) {
        if (_seOf(token) == token) return 0;
        // Buffered leg: free face balance is never book (SE shares are book).
        return bal; // <-- entire balance
    }
    uint256 book = Repo._layout().reserves[token];
    return bal > book ? bal - book : 0; // <-- correct, 2 lines below
}

```

`_refundFreeOfToken` then sends that to `to == msg.sender`, reached from `_refundConservation(msg.sender)` at 5 permissionless sites.

The comment's assumption — "free face balance is never book" — holds only if buffering is atomic and total. It is false in any frame where face token rests on the hook for another reason. Orbital is also the only one of the six hook families that makes **no attempt to buffer the balance back first**, and the only one that does not retain `MAX_DUST_WEI`.

PoC (hermetic, your own `TestBase_...OrbitalBufferHook`, production DFPkg deploy):

donation on hook	: 500 token0	
hook face (after)	: 0	<-- everything, not bal - MAX_DUST_WEI
attacker contributed	: 1 token0	
NET PROFIT	: 499 token0	

One ordinary `addLiquidity`. No privileged role, no race beyond being next caller.

Fix: make the buffered branch use the same `bal > book ? bal - book : 0` form as the branch two lines below it.

APEX-2026-009 — `_burnSeShares`, High

`ERC4626StandardExchangeCommon.sol:244` and a byte-identical twin at `MorphoBlueStandardExchangeCommon.sol:210`:

```

if (pretransferred) {
    ERC20Repo._burn(address(this), burnAmount);
    uint256 leftover = IERC20(address(this)).balanceOf(address(this));
    if (leftover > 0) IERC20(address(this)).safeTransfer(owner, leftover); // entire remainde
r
}

```

`owner` is `msg.sender` at all 6 call sites. `leftover` is the whole self-share balance with no booked-reserve subtraction and no link to what this caller pretransferred. Strictly worse than 003, which at least gates the refund behind `source == ShareSource.PublicBalanceRefundExcess`.

PoC: attacker holding **nothing** burns 1/1000 of the victim's position and receives the other 999/1000. Control test confirms the non-pretransferred path correctly burns from `msg.sender` and reverts.

`_receiptForExactShares` in the same file already does this correctly ~20 lines away:

```
uint256 prepaid = pretransferred ? _prepaidCredit(receipt, maximum) : 0;
if (prepaid > required) receipt.safeTransfer(msg.sender, prepaid - required);
```

APEX-2026-004B — idle underlying, High

`_refundOrAbsorbAbove(tokenIn, msg.sender, 0)` at `InTarget.sol:92,107` and `OutTarget.sol:88`. Its own docstring warns never to pass `keepBalance == 0` on tokens backing SE shares. The underlying is the documented-safe usage — but any idle underlying on the diamond still leaves with the next caller.

PoC: 250 UND from a third party resting on the SE → attacker contributes 10 UND via one ordinary `exchangeIn` → leaves with the full 250. SE left holding zero.

5. APEX-2026-005 — the root cause, and the one change that fixes it

All of the above are one defect: **unbooked balance treated as claimable**. There is no canonical helper, so it is implemented 10 times, 4 different ways, 2 of them unsafe.

You already wrote the correct version three times. In the *same file* as the broken one:

```
// UniswapV4StandardExchangeCommon.sol:1270  BROKEN — mixes stored R with live deployed
uint256 faceBooked = R > deployed ? R - deployed : 0;
uint256 U          = B0 > faceBooked ? B0 - faceBooked : 0;

// UniswapV4StandardExchangeCommon.sol:1313  CORRECT — ~30 lines below
uint256 R = MultiAssetBasicVaultRepo._reserveOfToken(address(this));
uint256 U = b0 > R ? b0 - R : 0;
```

`_secureShareDelivery` subtracts booked reserve and stops — and is right not to subtract `deployed`. The same correct/broken pair exists in the V3 file (`:907` vs `:855`).

Best of the three is `ERC4626StandardExchangeCommon._prepaidCredit:123`, which also clamps to the caller's own declared maximum:

```
uint256 available = token.balanceOf(address(this)) - MultiAssetBasicVaultRepo._reserveOfToken
(address(token));
return available < maximum ? available : maximum;
```

The entire bug is the extra `deployed` term. Delete it. Promoting `_prepaidCredit` to a shared library and routing all ten sites through it closes 001, 003, 004, 008 and 009 together.

Separately: dust that genuinely cannot be re-buffered should go to `feeTo`, not to the caller — the pattern `_absorbDustToFeeTo` already implements elsewhere in your codebase.

6. Retractions — two things I flagged that are NOT defects

I required an executed PoC for every finding before sending this. Two did not survive, and I am retracting them rather than leaving them on your plate.

`beforeSwap` missing `_onlyPoolManager()` — WITHDRAWN, false positive

I claimed `UniswapV4StandardExchangeBalancerQuadStableBufferHookHooksTarget.beforeSwap` lacked the guard its siblings have. **It does not.** Its first line is:

```
BeforeInitializeLib.beforeInitialize(key);
```

which itself begins `if (msg.sender != Repo._layout().poolManager) revert NotPoolManager();` plus validates currency ordering, token binding, the dynamic-fee flag, tick spacing and `poolKey.hooks == address(this)`. Strictly stronger than the guard I said was missing.

Confirmed empirically: a direct EOA call reverts `0xae18210a (NotPoolManager())`, and it is also rejected when called from inside an attacker-opened `PoolManager.unlock()` — the only context where `_take` / `_settle` would not trip the manager lock.

My error: I grepped for the guard's *name*, not its behaviour.

Weighted-hook dust refund — downgraded High → informational

I claimed any resting face balance on the weighted hook was claimable. **Measured, it is not.** The buffer-first branch converts the balance into SE shares held by the hook before anything is paid out, then re-reads:

```
donation placed on hook    : 500 token0
hook face balance (after)  : 10              (== MAX_DUST_WEI)
attacker face balance      : 0.000999       (its own unused join input)
attacker contributed       : 1 token0
=> donation buffered into SE shares held by the hook; accrues to LPs, NOT to the caller
```

No net profit. Same result for curve quad-stable, balancer quad-stable, single-CP and dual-CP. Two further attempts to reach the unsafe `preview == 0` branch naturally also failed.

Still worth a cleanup on principle: `to = msg.sender` is the wrong destination, and its safety depends entirely on a downstream call in a different contract fully consuming the balance. A future SE that under-consumes turns this into a real leak with no change to the hook files.

7. Coverage

Full-surface sweep, not a spot-check.

- **1,207** product contracts, **1,248/1,248** declared types, **12,009** functions mapped — 100%. Crane linked closure and test stubs also 100%.
- Value-**in** surface enumerated: 10 unbooked-credit implementations, 4 behaviours, 2 unsafe.
- Value-**out** surface enumerated: all **125** payout sites classified; all 20 unguarded "pays raw balance" sites triaged individually; all 5 payout-bearing callbacks verified guarded.
- **26 tests executed**: 16 hermetic green, 10 fork (9 green + the 1 intended invariant break).
- Math library: 9 properties × 200,000 runs = **1.8M**, zero counterexamples.
- Gate corner cases enumerated **exhaustively** (not fuzzed): 6 of 12 corners over-credit; every corner with `deployed >= R` credits the entire balance.

Limits I want to state plainly rather than paper over:

1. My broad composed-surface invariant is **vacuous** — 8 calls, 0 succeeded. It found nothing because it reached nothing. Do not read it as evidence.
2. The five buffer-first hook families are refuted only against `SimpleYieldERC4626`. An under-consuming SE would reopen the issue.
3. Symbolic execution (halmos) was inconclusive — 6 of 9 properties timed out on nonlinear `mulDiv` across two rounds at 8× budget. Substituted with the 1.8M-run fuzz above.
4. My CEI check across join paths was vacuous (mint and pull are split across a Target→Core delegation my tooling does not follow).

8. Suggested priority

1. **Now**: decide on 001-M. The DTF leg is open as of block 64025200 and rebalancing only closes it for minutes-to-hours. The durable fix is deleting the `deployed` term.
2. **Now**: 001-M2 — treat all 7 vaults, not 1.
3. **Now**: 003 on the live custody vault.
4. **Before next deploy**: 008 (orbital), 009 (`_burnSeShares` in both SEs), 004B.
5. **Structural**: 005 — promote `_prepaidCredit` to a shared helper; it subsumes the rest.

Happy to walk through any of the PoCs, or re-verify once you have patches up.